Horizon Capital

Technical Deep Dive

Version 0.2.0 | 2026-03-24

Contents

1 Data Sources & API Contracts

2 Data Processing Pipeline

3 Feature Engineering

4 ML & AI Models -- Choices, Alternatives & Reasoning

5 Trading Strategies

6 Risk & Portfolio Management

7 Project Timeline (Detailed)

8 Dependency & Architecture Decisions

* Research Papers Master List

Version 0.2.0 · Last updated: 2026-03-24

Table of Contents

- [Data Sources & API Contracts](#1-data-sources--api-contracts)
- [Data Processing Pipeline](#2-data-processing-pipeline)
- [Feature Engineering](#3-feature-engineering)
- [ML & AI Models -- Choices, Alternatives & Reasoning](#4-ml--ai-models--choices-alternatives--reasoning)
- [Trading Strategies](#5-trading-strategies)
- [Risk & Portfolio Management](#6-risk--portfolio-management)
- [Project Timeline (Detailed)](#7-project-timeline-detailed)
- [Dependency & Architecture Decisions](#8-dependency--architecture-decisions)

1. Data Sources & API Contracts

1.1 Yahoo Finance (`data_layer/yahoo_feed.py`)

Role: Primary historical OHLCV source for all equities.

Library: yfinance

Polling interval: 5 minutes

What We Call

```
yfinance.Ticker(symbol).history(period=period, interval=interval)
```

What We Receive -- Field Table

Raw Field	Our Field	Type	Notes
Open	open	float	Session open price
High	high	float	Session high
Low	low	float	Session low
Close	close	float	Adjusted close
Volume	volume	int	Shares/units traded
Datetime (index)	timestamp	datetime (UTC)	Converted to UTC
--	currency	str	Inferred from exchange suffix
--	exchange	str	Inferred from symbol suffix

Symbol Mapping We Perform

Internal Symbol	Yahoo Symbol	Exchange	Currency
RELIANCE.NSE	RELIANCE.NS	NSE	INR
TCS.BSE	TCS.BO	BSE	INR
AAPL.NASDAQ	AAPL	NASDAQ	USD
TSLA.NYSE	TSLA	NYSE	USD
BTC.CRYPTO	BTC-USD	CRYPTO	USD
HSBA.LSE	HSBA.L	LSE	GBP
7203.TSE	7203.T	TSE	JPY
000001.SSE	000001.SS	SSE	CNY

What We Discard

- Dividends, Stock Splits columns (not relevant for HFT-style signals)
- Pre-market / after-hours data (period filtered to market hours)

1.2 Alpaca Markets (`data_layer/alpaca_feed.py`)

Role: Live US equity ticks with sub-second latency.

Mode: Paper trading by default (settings.alpaca_paper_mode = True).

Endpoints Called

Endpoint	Method	Purpose
wss://stream.data.sandbox.alpaca.markets/v2/iex	WebSocket	Live tick stream (paper)
wss://stream.data.alpaca.markets/v2/iex	WebSocket	Live tick stream (live)
https://data.sandbox.alpaca.markets/v2/stocks/{symbol}/trades/latest	REST GET	Single latest trade (fallback)
https://data.sandbox.alpaca.markets/v2/stocks/{symbol}/bars	REST GET	OHLCV bars (fallback)

WebSocket Tick Message Fields

Raw JSON Key	Our Field	Type	Notes
S	symbol	str	e.g. "AAPL"
T	--	str	Message type ("t" = trade)
t	timestamp	datetime	ISO 8601 UTC
p	last_price	float	Last trade price
s	volume	int	Trade size (shares)
--	bid	float	None (not in Alpaca stream)
--	ask	float	None (not in Alpaca stream)
--	currency	str	Always "USD"

REST OHLCV Bar Fields

Raw JSON Key	Our Field	Type
t	timestamp	datetime
o	open	float
h	high	float
l	low	float
c	close	float
v	volume	int

Authentication Headers (REST)

```
APCA-API-KEY-ID: {settings.alpaca_api_key}
APCA-API-SECRET-KEY: {settings.alpaca_api_secret}
```

Fallback Logic

```
WebSocket OK? ??No??? REST poll every 10s ??No??? Disabled (circuit breaker)
      ?
     Yes
      ?
  Continuous ticks
```

1.3 Polygon.io (`data_layer/polygon_feed.py`)

Role: US equity OHLCV aggregates (free tier, ~15-min delay).

Polling interval: 60 seconds

Endpoints Called

Endpoint	Purpose
GET /v2/aggs/ticker/{symbol}/range/{mult}/{timespan}/{from}/{to}	OHLCV bars
GET /v2/last/trade/{symbol}	Most recent trade

OHLCV Bar Fields

Raw JSON Key	Our Field	Type	Notes
t	timestamp	datetime	Milliseconds -> UTC datetime
o	open	float	
h	high	float	
l	low	float	
c	close	float	
v	volume	int	
vw	--	float	VWAP (discarded -- computed ourselves)
n	--	int	Trade count (discarded)

Query Parameters We Set

Param	Value	Reason
adjusted	true	Dividend-adjusted prices
sort	asc	Oldest-first for feature computation
limit	200	Match our lookback window
timespan	minute	Finest free-tier granularity
multiplier	5	5-minute bars

1.4 CoinGecko (`data_layer/crypto_feed.py`)

Role: Free crypto spot prices -- no API key required.

Polling interval: 60 seconds

Endpoint Called

```
GET https://api.coingecko.com/api/v3/simple/price
  ?ids=bitcoin,ethereum,solana,...
  &vs_currencies=usd
  &include_24hr_vol=true
  &include_last_updated_at=true
```

Response Fields Per Coin

JSON Key	Our Field	Type	Notes
{coin}.usd	last_price	float	Spot price in USD
{coin}.usd_24h_vol	volume	float	24h volume in USD
{coin}.last_updated_at	timestamp	datetime	Unix -> UTC
--	open/high/low	float	Not available -- set to last_price (synthetic OHLCV)

Coin -> Internal Symbol Mapping

CoinGecko ID	Internal Symbol
bitcoin	BTC.CRYPTO
ethereum	ETH.CRYPTO
solana	SOL.CRYPTO
binancecoin	BNB.CRYPTO
ripple	XRP.CRYPTO
cardano	ADA.CRYPTO
dogecoin	DOGE.CRYPTO
polkadot	DOT.CRYPTO
avalanche-2	AVAX.CRYPTO
matic-network	MATIC.CRYPTO

Limitation: CoinGecko free tier gives only current price, no intraday OHLCV. We synthesize open = high = low = close = last_price for the bar record, and only use the close price in signal generation. Proper OHLCV would require CoinGecko Pro or Binance API.

1.5 Zerodha Kite (`data_layer/zerodha_feed.py`)

Role: Indian equity (NSE/BSE) tick data.

Status: Mock mode only (real KiteTicker not yet wired).

Mock Mode Behavior

Parameter	Value
Tick interval	2 seconds
Price random walk	+/-0.3% per tick
Spread simulation	5 basis points (bid = last - 0.025%, ask = last + 0.025%)
Initial seed prices	Hardcoded for 10 symbols (see table below)

Seed Prices (Mock Mode)

Symbol	Seed Price (INR)
RELIANCE.NSE	2,450
TCS.NSE	3,800
INFY.NSE	1,500
HDFCBANK.NSE	1,600
ICICIBANK.NSE	950
WIPRO.NSE	450
SBIN.NSE	620
BAJFINANCE.NSE	7,200

Symbol	Seed Price (INR)
LT.NSE	3,100
ITC.NSE	430

Real Mode (Planned -- KiteTicker)

```
# TODO in zerodha_feed.py:
# kite = KiteConnect(api_key=settings.zerodha_api_key)
# kws = KiteTicker(api_key, access_token)
# kws.on_ticks = self._on_kite_ticks
# kws.connect()
```

1.6 Forex Feed (`data_layer/forex_feed.py`)

Role: USD/INR rate for cross-currency P&L calculations.

Source: Yahoo Finance ticker USDINR=X

Polling interval: 5 minutes

Storage: Redis key horizon:forex:usdinr

1.7 News & Sentiment Sources

NewsAPI (`llm/newsapi_client.py`)

- Endpoint: GET https://newsapi.org/v2/everything?q={symbol}&language=en&sortBy=publishedAt
- Fields used: title, description, publishedAt, source.name
- Fields discarded: url, urlToImage, author, content (only first 200 chars in free tier)

Reddit PRAW (`llm/reddit_client.py`)

- Subreddits: r/investing, r/stocks, r/cryptocurrency, r/IndiaInvestments
- Fields used: title, selftext, score, num_comments, created_utc
- Engagement weight: score * log(1 + num_comments) -- upvoted posts with discussion weighted higher
- Limit: Top 25 posts per subreddit per call

2. Data Processing Pipeline

2.1 Full Pipeline (End-to-End)

```
Raw API Response
      ?
      ?
[Feed-specific parser]       ? Each feed has its own parser class
      ?
      ?
[Data Normalizer]            ? All feeds -> NormalizedTick / NormalizedOHLCV
      ?
      ?
[Circuit Breaker check]      ? Max 3 retries, exponential backoff
      ?
      ?
[SQLite / PostgreSQL write]  ? ohlcv table + ticks table
      ?
      ?
[Redis cache write]          ? horizon:tick:{symbol} -- TTL 5 min
      ?
      ?
[Feature Computation Worker] ? Every 5 minutes, reads last 200 bars
      ?
      ??? 23 Technical Indicators  (ta library)
      ??? Volatility Features      (realized vol, GARCH, VIX proxy)
      ??? Regime Feature Vector    (6D input for HMM)
            ?
            ?
      [Feature Store]              ? PostgreSQL features table + Redis cache
            ?
            ?
      [Strategy Agents]            ? Read features -> generate signals
            ?
            ?
      [Debate Engine]              ? Multi-agent consensus on trade
            ?
            ?
      [Execution Agent]            ? Place order (paper / real)
            ?
            ?
      [Performance Worker]         ? Track P&L, Sharpe, alpha, etc.
```

2.2 Normalized Schemas

NormalizedTick (single tick):

```
symbol:     str       # "AAPL", "RELIANCE", "BTC"
exchange:   str       # NSE | BSE | NASDAQ | NYSE | CRYPTO | LSE | ...
timestamp:  datetime  # UTC
last_price: float
bid:        float | None
ask:        float | None
volume:     int
currency:   str       # USD | INR | GBP | JPY | CNY | ...
```

NormalizedOHLCV (bar):

```
symbol:        str
exchange:      str
timestamp:     datetime   # UTC, bar open time
timeframe:     str        # 1m | 5m | 15m | 1h | 1d | 1w
open:          float
high:          float
low:           float
close:         float
volume:        int
open_interest: float | None   # Futures only
currency:      str
```

2.3 Database Tables Written

Table	Written By	Key Columns
ohlcv	DataIngestionWorker	symbol, timestamp, open, high, low, close, volume, exchange, currency
ticks	DataIngestionWorker	symbol, timestamp, last_price, bid, ask, volume, exchange, currency
features	FeatureComputationWorker	symbol, timestamp, features JSONB, version
signals	StrategyAgents	symbol, timestamp, direction, strength, strategy_name, regime
orders	ExecutionAgent	symbol, order_type, quantity, price, status
fills	ExecutionAgent	order_id, fill_price, fill_qty, timestamp
positions	PortfolioAgent	symbol, quantity, avg_cost, market_value
agent_states	Orchestrator	agent_id, status, last_heartbeat, metadata
debate_sessions	DebateEngine	session_id, signal_id, consensus_score, decision
debate_arguments	DebateEngine	session_id, role, argument, confidence, vote

3. Feature Engineering

3.1 Technical Indicators (23 total, `features/technical.py`)

All computed via ta library (pure Python -- not ta-lib which requires a C binary).

Trend (4 indicators)

Indicator	Parameters	Output Columns	Purpose
SMA	window=20	sma_20	Baseline trend direction
EMA	window=20	ema_20	Faster trend (more weight on recent prices)
Heiken-Ashi	--	ha_open, ha_high, ha_low, ha_close	Smoothed candles, noise reduction
Ichimoku	tenkan=9, kijun=26, senkou_b=52	tenkan, kijun, senkou_a, senkou_b, chikou	Full market structure in one indicator

Volatility (3 indicators)

Indicator	Parameters	Output Columns	Purpose
Bollinger Bands	window=20, std_dev=2.0	bb_upper, bb_middle, bb_lower, bb_bandwidth, bb_pct_b	Volatility envelope
ATR	window=14	atr_14	True range volatility, used in position sizing
StdDev	window=20	std_20	Baseline dispersion

Momentum (4 indicators)

Indicator	Parameters	Output Columns	Purpose
RSI	window=14, ob=70, os=30	rsi_14	Overbought/oversold oscillator
MACD	fast=12, slow=26, signal=9	macd_line, macd_signal, macd_hist	Trend + momentum crossover
Supertrend	window=10, multiplier=3.0	supertrend, supertrend_direction	Dynamic trailing trend
Stochastic	k=14, d=3	stoch_k, stoch_d	Momentum relative to range

Volume & Other (12 indicators)

Indicator	Output	Purpose
VWAP	vwap	Fair value intraday benchmark
OBV	obv	Volume flow direction
A/D	ad_line	Accumulation vs distribution
MFI	mfi_14	Volume-weighted RSI
CMF	cmf_20	Chaikin money flow
VROC	vroc_14	Volume rate of change
Parabolic SAR	psar, psar_bull, psar_bear	Dynamic trailing stop levels
Williams %R	williams_r	Overbought/oversold (range-based)
Awesome Oscillator	ao	Market momentum (Suri Duddella)
KDJ	k, d, j	Variation of stochastic (popular in Asia)
Elder's Force Index	efi	Price x volume force
Market Profile	poc, value_area_high, value_area_low	Auction theory price levels

3.2 Volatility Features (`features/volatility.py`)

Feature	Formula	Purpose
Realized Vol (20d)	std(log_returns, 20) * sqrt252	Historical annualized vol
GARCH(1,1) Forecast	arch_model(returns, vol="Garch", p=1, q=1) -- 1-day ahead	Forward-looking conditional vol
VIX Proxy	std(log_returns, 30) * sqrt252 * 100	Implied-vol substitute (NSE/BSE have no VIX feed)
GARCH params	omega, alpha, beta	Model diagnostics

3.3 Regime Feature Vector (`features/regime_features.py`)

6-dimensional input vector for HMM:

Dimension	Formula	Why
1. Log returns	log(close_t / close_{t-1})	Core signal
2. 20d rolling vol	std(returns, 20)	Short-term volatility
3. 60d rolling vol	std(returns, 60)	Medium-term volatility
4. Vol of vol	std(rolling_vol_20d, 20)	Regime instability indicator
5. 20d skewness	skew(returns, 20)	Tail asymmetry (crash signal)
6. 20d kurtosis	kurt(returns, 20)	Fat tails (crisis signal)

NaN rows are dropped before feeding to HMM.

4. ML & AI Models -- Choices, Alternatives & Reasoning

4.1 Regime Detection -- Why Hidden Markov Model?

Task

Classify the current market into one of 4 states: BULL, BEAR, HIGH_VOL, CRISIS

Alternatives Considered

Model	Pros	Cons	Citation
Gaussian HMM [x] (chosen)	Probabilistic states, unsupervised, captures latent structure, posterior confidence	Markov assumption (limited memory), sensitive to initial params	Hamilton (1989)
Markov-Switching ARMA	More general mean dynamics	Computationally expensive, overparameterized for daily regime	Hamilton (1994)
LSTM Classifier	Long memory, non-linear	Requires labeled regimes (chicken-and-egg), overfits on finance	--
K-Means Clustering	Simple, fast	Hard assignments, no temporal dependency, no confidence	--
Threshold-based Rules	Transparent, fast	Rigid, no uncertainty, misses transitions	--
Regime-Switching GARCH	Combines vol + regime	Very slow fitting (~50x slower), requires MCMC	Gray (1996)
Kalman Filter	Continuous states, linear	Doesn't model discrete regimes well	--

Why HMM Won

- Unsupervised: No need to hand-label "this was a BULL period" -- the model finds regimes from returns statistics alone.
- Probabilistic output: Returns posterior probability per state -- we use this as confidence score. Rules don't give this.
- Temporal smoothing: The Viterbi / forward-backward algorithm enforces temporal consistency. A single volatile day doesn't flip regime.
- 4 states maps to reality: Bull (trending up), Bear (trending down), High-Vol (choppy), Crisis (severe drawdown) -- all empirically present in NSE/BSE and US markets.
- Academic validation: Hamilton (1989) has ~14,000 citations. Guidolin & Timmermann (2007) specifically validate 4-state models for asset allocation.

Why Not 2 States?

A 2-state HMM (Bull/Bear) misses the HIGH_VOL state -- periods like Feb 2018, Sep 2022 where markets are not clearly

trending but are highly volatile. Different strategies apply in those periods vs. a bear trend.

Model Configuration

```
GaussianHMM(
    n_components=4,      # 4 regimes
    covariance_type="full",  # Full covariance between features
    n_iter=200,          # EM iterations
    random_state=42
)
```

Input: (T, 2) matrix -- [returns, 20d rolling vol]

Output: regime label + posterior probability array

Papers:

- Hamilton, J.D. (1989). "A New Approach to the Economic Analysis of Nonstationary Time Series and the Business Cycle." *Econometrica*, 57(2), 357-384. (~14,000 citations)
- Guidolin, M., & Timmermann, A. (2007). "Asset Allocation under Multivariate Regime Switching." *Journal of Economic Dynamics and Control*, 31(11), 3503-3544.
- Ang, A., & Timmermann, A. (2012). "Regime Changes and Financial Markets." *Annual Review of Financial Economics*, 4, 313-337.

4.2 Sentiment Analysis -- Why FinBERT + Loughran-McDonald?

Task

Score financial text (news headlines, Reddit posts, earnings calls) as positive / negative / neutral.

Alternatives Considered

Model	F1 on FinPhraseBank	F1 on Earnings	Speed	Cons
FinBERT [x] (primary, 70%)	97.0%	86%	Medium	Needs GPU for bulk
VADER	63%	58%	Very fast	General-purpose -- wrong for finance ("liability" = negative in VADER but neutral legally)
TextBlob	61%	54%	Fast	Same problem -- generic
Harvard General Inquirer	68%	60%	Fast	General vocabulary
Loughran-McDonald [x] (secondary, 30%)	73%	82%	Fast	Dictionary only, no context
FinBERT-FOMC	N/A	91%	Medium	Too narrow -- Fed-focused only
RoBERTa-Financial	96.5%	84%	Slower	Marginally worse than FinBERT
GPT-4 (zero-shot)	~89%	~91%	Slow + expensive	High latency, cost scales with volume
BERT-base (fine-tuned)	89.2%	77%	Medium	Less domain-specific than FinBERT

Why FinBERT Won

Loughran & McDonald (2011, ~6,000 citations) showed that the Harvard General Inquirer misclassifies 75% of financial terms. For example, "liability" is flagged as negative by general dictionaries but is standard neutral legal terminology in finance. FinBERT was pre-trained on 4.9 billion tokens of financial text (Reuters, Bloomberg, SEC filings) and fine-tuned on the Financial PhraseBank.

Combined Score:

```
final = 0.70 * finbert_score + 0.30 * loughran_mcdonald_score
```

The ensemble beats either individual model because:

- FinBERT handles context and syntax well but can hallucinate on very short text
- L-M dictionary is exact on known financial terms even on 1-sentence headlines

Loughran-McDonald Dictionary Categories Used:

Category	Size	Example Terms
Positive	~2,000	growth, improve, opportunity, gain, profit, success, strong, exceed
Negative	~2,000	decline, loss, failure, risk, crisis, bankruptcy, weak, miss
Uncertainty	~500	assume, estimate, perhaps, possible, unclear, uncertain
Litigious	~300	lawsuit, litigation, legal, defendant, verdict, claims

Papers:

- Araci, D. (2019). "FinBERT: Financial Sentiment Analysis with Pre-trained Language Models." *arXiv:1908.10063*. (~778 citations)
- Loughran, T., & McDonald, B. (2011). "When is a Liability not a Liability? Textual Analysis, Dictionaries, and 10-Ks." *Journal of Finance*, 66(1), 35-65. (~6,000 citations)
- Malo, P., et al. (2014). "Good Debt or Bad Debt: Detecting Semantic Orientations in Economic Texts." *JASIST*, 65(4), 782-796. (Financial PhraseBank dataset)

4.3 Volatility Forecasting -- Why GJR-GARCH(1,1)?

Task

Forecast next-day conditional volatility for position sizing and VaR.

Alternatives Considered

Model	Forecast Accuracy (QLIKE)	Captures Leverage?	Heavy Tails?	Complexity
GJR-GARCH(1,1) + Student-t [x]	Best	Yes	Yes	Low
GARCH(1,1) + Normal	Second-best	No	No	Very low
EGARCH(1,1)	Competitive	Yes	Partial	Medium
HAR-RV	Good (daily)	No	No	Medium
Realized GARCH	Very good	Yes	Yes	High
LSTM Volatility	Competitive	Implicit	Yes	Very high
Historical Vol (20d rolling std)	Worst	No	No	Trivial
EWMA (RiskMetrics)	Moderate	No	No	Low
Stochastic Vol (SV-t)	Very good	Yes	Yes	Very high (MCMC)

Why GJR-GARCH Won

Hansen & Lunde (2005, ~3,000 citations) compared 330 volatility models against GARCH(1,1) on 30 IBM stocks. No model significantly beat GARCH(1,1) on out-of-sample forecasting -- confirming the parsimony principle. The GJR extension captures the leverage effect (negative return shocks increase volatility more than positive shocks of equal magnitude) which is critical for equity markets.

GJR-GARCH(1,1) Equation:

```
sigma^2_t = omega + alpha·epsilon^2_{t-1} + gamma·I_{t-1}·epsilon^2_{t-1} + beta·sigma^2_{t-1}
```

- I_{t-1} = 1 if epsilon_{t-1} < 0 (leverage indicator)
- gamma > 0 confirms leverage effect -- standard finding in equities
- Student-t innovations: captures fat tails (kurtosis > 3 in daily returns)

Why not LSTM?

- Needs 1,000+ samples for reliable training; our symbols may have 200 bars
- Black box -- can't extract alpha, beta for risk reports
- Not interpretable in a regulatory context

Papers:

- Hansen, P.R., & Lunde, A. (2005). "A Forecast Comparison of Volatility Models: Does Anything Beat a GARCH(1,1)?" *Journal of Applied Econometrics*, 20(7), 873-889. (~3,000 citations)
- Corsi, F. (2009). "A Simple Approximate Long-Memory Model of Realized Volatility." *Journal of Financial Econometrics*, 7(2), 174-196. (~2,100 citations)
- Glosten, L.R., Jagannathan, R., & Runkle, D.E. (1993). "On the Relation between the Expected Value and the Volatility of the Nominal Excess Return on Stocks." *Journal of Finance*, 48(5), 1779-1801. (Original GJR paper)

4.4 Value-at-Risk -- Why GARCH-EVT?

Task

Estimate portfolio loss at 95% and 99% confidence levels.

Alternatives Considered

Method	Captures Time-Varying Vol?	Models Tail Correctly?	Backtests at 99%?	Notes
GARCH-EVT (McNeil & Frey) [x]	Yes	Yes (GPD)	Best	Gold standard
Historical Simulation (250d)	No	Implicit	Poor	Ignores volatility clustering
Parametric Normal	Yes (GARCH)	No (thin tails)	Fails	2008 underestimates
Filtered Historical Simulation	Yes	Implicit	Good	No extreme tail fit
Monte Carlo (Normal)	Yes	No	Poor	Same tail problem
Cornish-Fisher Expansion	Partial	Approximate	Moderate	Quick but imprecise
Student-t VaR	No	Yes	Moderate	Static DOF

GARCH-EVT 4-Step Process (McNeil & Frey 2000)

```
Step 1: Fit GJR-GARCH(1,1)-t to returns
Step 2: Extract standardized residuals z_t = r_t / sigma_t
Step 3: Fit GPD to left tail of {z_t} (threshold = 25th percentile)
Step 4: VaR = sigma_{t+1} x quantile_GPD(p)
        CVaR = sigma_{t+1} x E[z | z > quantile_GPD(p)]
```

Kuester, Mittnik & Paolella (2006, ~1,200 citations) benchmarked 14 VaR methods on 1,600 equity time series. GARCH-EVT ranked first on the Basel traffic-light test and coverage ratio. Historical simulation ranked last.

Papers:

- McNeil, A.J., & Frey, R. (2000). "Estimation of Tail-Related Risk Measures for Heteroscedastic Financial Time Series." *Journal of Empirical Finance*, 7(3-4), 271-300. (~2,500 citations)

- Rockafellar, R.T., & Uryasev, S. (2000). "Optimization of Conditional Value-at-Risk." *Journal of Risk*, 2(3), 21-41. (~7,000 citations)
- Kuester, K., Mittnik, S., & Paolella, M.S. (2006). "Value-at-Risk Prediction: A Comparison of Alternative Strategies." *Journal of Financial Econometrics*, 4(1), 53-89. (~1,200 citations)

4.5 Concept Drift -- Why LFR Detector?

Task

Detect when the HMM regime model has degraded (market dynamics changed, model predictions are wrong).

Alternatives

Method	Strengths	Weaknesses
LFR (Lu et al. 2019) [x]	Tracks 4 performance rates jointly, statistically grounded	Requires labeled stream
ADWIN	Parameter-free, works on any stream	High false-positive rate
DDM (Gama 2004)	Simple, classic	Only tracks error rate, misses precision/recall imbalance
PHT (Drift via cusum)	Robust to noise	Laggy
Page-Hinkley	Simple	Only detects one-directional drift

LFR tracks TPR, TNR, PPV, NPV simultaneously using EWMA (beta=0.9). A decline in any rate below its control bound triggers WARNING or DRIFT. This is important because a regime model can remain accurate on BULLs but silently degrade on BEAR detection -- single error rate won't catch that.

Paper:

- Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., & Zhang, G. (2019). "Learning under Concept Drift: A Review." *IEEE Transactions on Knowledge and Data Engineering*, 31(12), 2346-2363. (~1,500 citations)

4.6 Portfolio Sizing -- Why Half-Kelly?

Task

Determine position sizes given expected return and risk.

Alternatives

Method	Growth Rate	Drawdown	Practicality
Full Kelly	Maximum long-run	Very high (~50%)	Dangerous in practice
Half-Kelly [x]	~75% of full	~25% of full	Used by professionals
Fixed fractional (1-5%)	Low but stable	Low	Industry standard
Equal weight	N/A	Medium	Simple but no edge
Mean-Variance (Markowitz)	Theoretically optimal	Depends	Sensitive to estimation error

MacLean, Thorp & Ziemba (2011) showed that Half-Kelly achieves ~75% of the maximum geometric growth while cutting drawdowns by ~50%. Full Kelly leads to ruin with probability 1 if the expected return estimate is even slightly wrong (negative Kelly f* from overestimation).

Kelly Formula:

```
f* = (p x b - q) / b
position_size = (f* x 0.5) x portfolio_value
```

Where p = win_rate, b = avg_win / avg_loss, q = 1 - p.

Paper:

- MacLean, L.C., Thorp, E.O., & Ziemba, W.T. (2011). "The Kelly Capital Growth Investment Criterion." *World Scientific*.

4.7 Covariance Estimation -- Why Ledoit-Wolf Shrinkage?

Task

Estimate correlation matrix for portfolio diversification (position correlation check).

Problem With Sample Covariance

With N assets and T observations, if T < N^2, the sample covariance matrix is unstable -- small-sample error in individual covariances blows up portfolio optimization.

Ledoit-Wolf Shrinkage

```
Sigma? = (1 - alpha) x Sigma_sample + alpha x (trace(Sigma_sample)/N) x I
```

Shrinks toward scaled identity -- effectively assuming assets are less correlated than they appear in small samples.

Why this works: Stein (1956) showed that the MLE for the covariance matrix is inadmissible (dominated) under squared-error loss. Shrinkage estimators dominate in out-of-sample performance. Ledoit & Wolf (2004, ~3,500 citations) provide the optimal closed-form shrinkage intensity alpha.

Paper:

- Ledoit, O., & Wolf, M. (2004). "Honey, I Shrunk the Sample Covariance Matrix." *Journal of Portfolio Management*, 30(4), 110-119. (~3,500 citations)

5. Trading Strategies

5.1 Regime-Strategy Routing

```
HMM Output
    ?
    ??? BULL ?????????????? Momentum Strategy
    ??? BEAR ?????????????? Mean Reversion Strategy
    ??? NEUTRAL ??????????? Mean Reversion Strategy
    ??? HIGH_VOL ?????????? Volatility Strategy
    ??? CRISIS ????????????? Crisis Defense Strategy
```

5.2 Momentum Strategy -- Indicator Fusion Weights

Component	Weight	Signal Condition
RSI	25%	RSI < 30 -> strong buy; RSI > 70 -> sell
MACD Crossover	30%	MACD > Signal line
VPVMA	20%	Volume-adjusted MACD
EMA Trend	15%	Close > EMA_20
Volume Confirmation	5%	Volume > 20-day avg
ATR	5%	ATR expansion (volatility momentum)

Entry: Confluence score > 0.5 (majority of indicators agree)

Paper: Jegadeesh & Titman (1993, ~12,000 citations) -- cross-sectional momentum works over 3-12 month horizons. Multi-indicator fusion (arXiv 2206.12282) achieves 0.84-0.86 win-rate on US equities.

5.3 Mean Reversion -- OU Process Calibration

Ornstein-Uhlenbeck calibrated via maximum likelihood:

```
kappa = -log(rho) / Deltat          # mean reversion speed
theta = mu_x / (1 - rho)          # long-term mean
sigma = std_x x sqrt(1 - rho^2)   # OU volatility
```

s-score entry threshold of 1.25 comes directly from Avellaneda & Lee (2010), who derived it as optimal for PCA-based statistical arb on US equities with Sharpe 1.44.

5.4 Crisis Defense -- Why Trend-Following in Crisis?

Counterintuitively, trend-following (momentum) works inside a crisis because markets trend strongly downward with sporadic relief rallies. Bhansali & Davis (2010) showed that defensive put options cost too much in normal times (negative expected value) while trend-following is zero-cost and captures "crisis alpha." Harvey et al. (2018) showed volatility-targeting cuts left-tail loss by ~30% without sacrificing return.

6. Risk & Portfolio Management

6.1 Risk Parameters Summary

Parameter	Value	Source / Rationale
Max position per symbol	5% of NAV	Industry standard for non-concentrated books
Max drawdown trigger	20%	Typical institutional drawdown covenant
VaR confidence	95% + 99%	Basel III requires both levels
Kill switch: VaR breach	Portfolio x 2%	Empirical threshold for daily loss limit
Kill switch: correlation spike	> 0.85	Near-unity correlation = no diversification
Kill switch: spread > 0.5%	bid-ask spread	Illiquidity proxy
Kelly multiplier	0.5 (half)	MacLean, Thorp & Ziemba recommendation
GARCH min samples	60	Asymptotic efficiency for MLE
Mean reversion min samples	30	OU calibration stability

7. Project Timeline (Detailed)

Phase 1 -- Foundation (Completed)

Goal: Core architecture, agent scaffold, local-first storage

Key deliverables:

- BaseAgent class with lifecycle (start/stop/heartbeat)
- AgentBus message passing (in-memory for LOCAL, Redis for POSTGRES)
- LocalStorage -- 23-table SQLite schema
- Runtime system -- get_bus(), get_session_factory(), lazy imports

- Settings -- Pydantic config with .env support
- Orchestrator -- starts/stops agents, monitors heartbeats
- CLI entry point (__main__.py)

Architecture decision: In-memory bus uses Python asyncio.Queue -- zero dependencies. Redis bus uses PubSub. Same AgentBus interface, different backend. This let us test everything without Docker.

Phase 2 -- Data Layer (Completed)

Goal: Real market data flowing into the system

Key deliverables:

- DataNormalizer -- canonical NormalizedTick / NormalizedOHLCV schemas
- Feed implementations: Yahoo Finance, Alpaca, Polygon, CoinGecko, Zerodha (mock)
- ForexFeed -- USD/INR rate
- DataIngestionWorker -- polls all feeds, writes to DB, emits on bus
- Redis tick cache (horizon:tick:{symbol}, TTL 5 min)
- Circuit breaker on each feed (3 retries, exponential backoff)

Technical decision: Use ta (pure Python) instead of ta-lib (C binary). ta-lib requires system-level C libraries -- impossible to install reliably on Windows 11 without Visual Studio Build Tools. ta provides all 23 indicators we need with identical API, zero C dependencies.

Phase 3 -- Feature Engineering (Completed)

Goal: Transform raw OHLCV into model-ready features

Key deliverables:

- TechnicalFeatures -- 23 indicators via ta library
- VolatilityFeatures -- realized vol, GJR-GARCH(1,1), VIX proxy
- RegimeFeatures -- 6D vector: returns + vols + vol-of-vol + skew + kurtosis
- FeatureStore -- write to PostgreSQL features table + Redis cache
- FeatureComputationWorker -- runs every 5 minutes per symbol
- FeatureMessage published on bus so agents know when to refresh

Phase 4 -- ML Models & Regime Detection (Completed)

Goal: Classify market regimes and generate trading signals

Key deliverables:

- GaussianHMM wrapper -- 4-state model, train/predict/save/load
- RegimeAgent -- runs HMM, publishes regime + confidence every 5 min
- LFRDriftDetector -- detects HMM degradation
- FinBERTScorer + LoughranMcDonaldDictionary -- 70/30 sentiment ensemble
- SentimentAgent -- aggregates news + Reddit -> per-symbol sentiment score
- ModelRegistry -- version models, track performance

Phase 5 -- Strategies, Debate & Risk (Completed)

Goal: Generate, debate, and execute trading signals

Key deliverables:

- 4 strategies: MomentumStrategy, MeanReversionStrategy, VolatilityStrategy, CrisisStrategy
- EnsembleSelector -- routes regime -> appropriate strategy
- DebateEngine -- 4-agent LLM debate (Strategy, Risk, Sentiment, Portfolio roles)

- ClaudeReasoner -- Anthropic API integration with retry
- VaRCalculator -- GARCH-EVT with GPD tail fit
- KillSwitch -- liquidates on threshold breach
- PortfolioOptimizer -- Kelly + Ledoit-Wolf shrinkage
- ExecutionAgent -- paper order routing (mock fills)

Phase 6 -- Advanced Services (In Progress)

Goal: Production-grade reliability and observability

Key deliverables:

- StressTesting -- 5 historical crisis scenarios
- AlertManager -- rule-based alerts (console, Telegram, Slack, email)
- ModelRegistry -- Thompson Sampling A/B testing for new models
- ReportGenerator -- daily/weekly/monthly/quarterly reports (Rich + PDF)
- HealthMonitor -- agent heartbeats, DB health, feed status, LLM availability

Phase 7 -- Backtesting Engine (Planned)

Goal: Rigorous strategy evaluation on historical data

Planned deliverables:

- Event-driven backtester (not vectorized -- simulates real execution latency)
- Slippage model (bid-ask spread from actual data)
- Per-strategy walk-forward optimization
- Benchmark comparison (NIFTY 50, S&P 500)
- Overfitting guard: Deflated Sharpe Ratio (Bailey & Lopez de Prado 2014)

Phase 8 -- Production Brokers (Planned)

Goal: Real money execution

Planned deliverables:

- Zerodha KiteTicker real integration (NSE/BSE live ticks + order routing)
- Alpaca live mode (US equities)
- Binance API (crypto execution -- replace CoinGecko read-only)
- FIX protocol adapter (institutional)
- Order management system (OMS) with fills reconciliation

Phase 9 -- ML Automation (Planned)

Goal: Self-improving system

Planned deliverables:

- Continuous HMM retraining (daily, walk-forward)
- Hierarchical Risk Parity (HRP) -- replace Markowitz (Lopez de Prado 2016)
- Reinforcement Learning policy for position sizing (PPO or SAC)
- Feature importance tracking (SHAP values)
- Automatic strategy parameter tuning (Optuna)

Phase 10 -- Institutional Features (Planned)

Goal: Hedge fund operational readiness

Planned deliverables:

- Multi-currency accounting (INR + USD + GBP P&L aggregation)
- Tax optimization (STCG / LTCG tracking for Indian equities)
- Audit trail for all trade decisions (debate arguments stored permanently)
- LP reporting dashboard
- Compliance checks (SEBI regulations for AIF Category III)

8. Dependency & Architecture Decisions

8.1 Why `ta` Instead of `ta-lib`

Factor	ta (pure Python)	ta-lib (C library)
Install on Windows	pip install ta [x]	Requires C compiler, Visual Studio [ ]
Install on Linux	pip install ta [x]	apt install libta-lib-dev then pip
Performance	Adequate (pandas vectorized)	2-5x faster on large arrays
API compatibility	Pandas-native	Numpy arrays only
Indicator count	130+	150+
Decision	[x] Chosen	Rejected (install complexity)

8.2 Why SQLite (LOCAL mode) Instead of PostgreSQL

LOCAL mode targets a developer's laptop with zero infrastructure setup. SQLite:

- Single file database, no server
- Async support via aiosqlite
- Sufficient for < 10M rows (years of daily OHLCV for 100 symbols)
- Identical SQLAlchemy ORM interface -- swap to PostgreSQL with one env var change

8.3 Why `hmmlearn` Instead of Custom HMM

hmmlearn is the de facto Python HMM library (used in scikit-learn ecosystem), implements Baum-Welch EM via Cython (fast), and is well-tested. Writing a custom Gaussian HMM from scratch would be error-prone in the EM M-step, especially for the full covariance update.

8.4 Why `arch` Instead of `statsmodels` for GARCH

arch (Kevin Sheppard, Oxford) is the gold standard for GARCH estimation in Python:

- Supports GJR-GARCH, EGARCH, HAR-RV, and 10+ other volatility models
- Robust maximum likelihood estimation (convergence guarantees)
- Forecasting with bootstrap confidence intervals
- statsmodels GARCH implementation is older, has known convergence issues

8.5 Why Multi-Agent Debate Instead of Simple Signal Average

Simple averaging of signals treats all agents equally and produces no reasoning. The debate architecture:

- Forces explicit reasoning -- each debater must articulate why it votes FOR/AGAINST
- Role specialization -- risk manager will always raise tail scenarios; momentum trader will focus on trend strength
- Audit trail -- all arguments stored in DB for post-trade analysis
- LLM integration natural -- each debater is one LLM call with a role-specific system prompt
- Regulatory alignment -- "we can explain every trade" because the debate transcript is the explanation

8.6 Why Python 3.11+ Syntax (`X | None` Not `Optional[X]`)

Ruff UP007 rule: Optional[X] is deprecated since Python 3.10 in favor of X | None. The codebase targets Python 3.11+ (syntax X | Y union types are native, no from __future__ import annotations needed). This keeps imports clean and aligns with modern Python typing.

Research Papers Master List

Paper	Year	Citations	Used In
Hamilton -- Markov-Switching	1989	~14,000	HMM regime detection
Engle & Granger -- Cointegration	1987	~30,000	Mean reversion (Nobel Prize)
Jegadeesh & Titman -- Momentum	1993	~12,000	Momentum strategy
Rockafellar & Uryasev -- CVaR	2000	~7,000	VaR/CVaR calculation
Loughran & McDonald -- Finance Dictionary	2011	~6,000	Sentiment (L-M 30% weight)
DeMiguel et al. -- 1/N Portfolio	2009	~6,000	Portfolio optimization baseline
Ledoit & Wolf -- Shrinkage	2004	~3,500	Covariance estimation
Hansen & Lunde -- GARCH comparison	2005	~3,000	GARCH model selection
Corsi -- HAR-RV	2009	~2,100	Realized volatility
McNeil & Frey -- GARCH-EVT	2000	~2,500	VaR calculation
Gatev et al. -- Pairs Trading	2006	~2,000	Mean reversion pairs
Avellaneda & Lee -- Stat Arb	2010	~800	OU s-score thresholds
Kuester et al. -- VaR comparison	2006	~1,200	VaR method selection
Lu et al. -- Concept Drift Survey	2019	~1,500	Drift detection
Lopez de Prado -- HRP	2016	~1,500	Planned: portfolio optimization
Araci -- FinBERT	2019	~778	Sentiment analysis (70% weight)
Harvey et al. -- Vol Targeting	2018	~400	Crisis strategy
Bhansali & Davis -- Crisis Alpha	2010	~200	Crisis defense strategy
MacLean, Thorp & Ziemba -- Kelly	2011	~1,000	Position sizing
Guidolin & Timmermann -- 4-State HMM	2007	~600	4-regime validation

This document is auto-generated from codebase analysis. For source code references, see the files in horizon-capital/backend/app/.